

《Attention Is All You Need》 Transformer 模型复现实验报告

一、实验题目

基于 PyTorch 的 Transformer 模型复现与关键结构分析

二、实验目的

本实验旨在复现 Vaswani 等人在论文《Attention Is All You Need》中提出的 Transformer 模型，并通过英德翻译任务理解 Transformer 的核心结构和工作机制。

本实验主要目标包括：

- 使用 PyTorch 从零实现 Transformer Encoder-Decoder 模型。
- 理解 Scaled Dot-Product Attention 和 Multi-Head Attention 的计算过程。
- 理解 Q、K、V 分离设计在注意力机制中的作用。
- 理解位置编码对 Transformer 序列建模能力的影响。
- 完成从数据加载、分词、模型训练到翻译推理的完整流程。
- 通过对比实验分析 K/V 合并、不使用位置编码、简单绝对位置编码等结构变化对模型的影响。

三、实验环境

深度学习框架 PyTorch

数据集 Hugging Face opus_books

数据集配置 de-en

任务类型 English to German 机器翻译

分词方法 BPE

优化器 Adam

学习率策略 Noam Scheduler

解码方式 Greedy Decoding

四、论文方法概述

Transformer 是一种完全基于注意力机制的序列到序列模型。与传统 RNN、LSTM 或 GRU 不同, Transformer 不依赖循环结构, 而是通过 Self-Attention 直接建模序列中任意两个 token 之间的关系。

Transformer 主要由 Encoder 和 Decoder 两部分组成。

Encoder 用于将源语言句子编码成上下文表示; Decoder 根据 Encoder 的输出和已经生成的目标语言 token, 逐步生成目标语言句子。

原论文中的 Transformer Base 配置如下:

参数	设置
Encoder 层数	6
Decoder 层数	6
d_model	512
注意力头数	8
前馈网络隐藏层维度	2048
Dropout	0.1
Label smoothing	0.1
Optimizer	Adam
Adam betas	0.9, 0.98
Adam eps	1e-9
Warmup steps	4000

本实验基本按照 Transformer Base 的结构进行实现, 但考虑到设备和训练时间限制, 数据集采用较小规模的 opus_books 英德平行语料, 而不是论文中的 WMT14 English-German 数据集。

五、Transformer 核心结构

5.1 Scaled Dot-Product Attention

Transformer 的核心计算是 Scaled Dot-Product Attention, 其公式为:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$$

其中:

- Q 表示 Query;
- K 表示 Key;
- V 表示 Value;

- d_k 表示 Key 向量的维度。

该公式首先计算 Q 和 K 的点积，得到 token 之间的相关性分数；然后除以 $\sqrt{d_k}$ ，避免点积结果过大导致 softmax 梯度过小；最后通过 softmax 得到注意力权重，并对 V 进行加权求和。

5.2 Multi-Head Attention

Multi-Head Attention 将 Q、K、V 投影到多个不同的子空间中，并行计算多个注意力头。每个注意力头可以关注不同类型的依赖关系。

例如，在机器翻译任务中：

- 某些 head 可能关注主谓关系；
- 某些 head 可能关注动宾关系；
- 某些 head 可能关注长距离依赖；
- 某些 head 可能关注局部短语结构。

多个 head 的结果会被拼接起来，再经过一个线性层得到最终输出。

5.3 位置编码

由于 Transformer 没有 RNN 的循环结构，也没有 CNN 的局部卷积结构，因此模型本身无法天然感知 token 的顺序。为了解决这一问题，原论文引入了位置编码。

正弦余弦位置编码公式为：

$$PE(pos, 2i) = \sin(pos / 10000^{(2i / d_{model})})$$

$$PE(pos, 2i+1) = \cos(pos / 10000^{(2i / d_{model})})$$

其中：

- pos 表示 token 在序列中的位置；
- i 表示维度索引；
- d_{model} 表示模型隐藏层维度。

位置编码与 token embedding 相加后输入模型，使模型既能获得词语语义信息，也能获得词语位置信息。

5.4 前馈神经网络

每个 Encoder Layer 和 Decoder Layer 中都包含一个前馈神经网络：

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

该网络由两个线性层组成，中间使用 ReLU 激活函数。它对每个位置的表示独立进行非线性变换，提高模型表达能力。

5.5 残差连接和层归一化

Transformer 每个子层之后都使用残差连接和 LayerNorm：

$$\text{LayerNorm}(x + \text{Sublayer}(x))$$

残差连接可以缓解深层网络训练时的梯度消失问题，LayerNorm 可以稳定训练过程，使模型更容易收敛。

六、模型实现

本实验项目结构如下：

```
F:\prml-4
├── model.py
├── data.py
├── utils.py
├── train.py
└── translate.py
```

各文件功能如下：

文件	功能
model.py	定义 Transformer 模型结构
data.py	加载数据集、训练 tokenizer、构造 DataLoader
utils.py	实现 NoamScheduler 和 label smoothing loss
train.py	模型训练和 checkpoint 保存
translate.py	模型加载和翻译推理

6.1 MultiHeadAttention 实现

在 `MultiHeadAttention` 中，输入首先通过三个线性层生成 Q、K、V：

```
q = self.w_q(q)
k = self.w_k(k)
v = self.w_v(v)
```

然后将它们拆分为多个 head，并计算注意力分数：

```
scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(self.d_k)
```

如果存在 mask，则对无效位置进行屏蔽：

```
scores = scores.masked_fill(mask == 0, -1e9)
```

之后通过 softmax 得到注意力权重，并与 V 相乘：

```
attn = torch.softmax(scores, dim=-1)
out = torch.matmul(attn, v)
```

6.2 EncoderLayer 实现

`EncoderLayer` 包括两个子层：

Multi-Head Self-Attention
Feed Forward Network

其结构为：

```
x = LayerNorm(x + SelfAttention(x))
x = LayerNorm(x + FeedForward(x))
```

6.3 DecoderLayer 实现

`DecoderLayer` 包括三个子层：

Masked Multi-Head Self-Attention
Encoder-Decoder Cross Attention
Feed Forward Network

其中 Masked Self-Attention 用于防止 Decoder 看到未来 token，Cross Attention 用于让 Decoder 关注 Encoder 的输出。

6.4 Mask 机制

本实验实现了两类 mask。

第一类是 padding mask，用于屏蔽 [PAD] 位置，避免模型关注无效 token。

第二类是 subsequent mask，用于 Decoder 自注意力层，保证模型在预测当前位置时只能看到当前位置之前的信息，不能看到未来 token。

6.5 Label Smoothing

本实验使用 label smoothing 损失函数，平滑系数为 0.1。它可以避免模型对正确类别过度自信，提高泛化能力。

6.6 Noam 学习率调度

本实验实现了原论文中的 Noam 学习率调度策略：

$$lr = d_model^{(-0.5)} * \min(step^{(-0.5)}, step * warmup_steps^{(-1.5)})$$

该策略在训练初期逐渐增大学习率，经过 warmup 后逐渐降低学习率，有助于 Transformer 稳定训练。

七、推理结果

训练完成后，运行：

```
python translate.py
```

输入测试句子：

I love machine learning.

模型输出：

Ich habe sie nicht .

该结果表明模型已经能够根据英文输入生成德语 token 序列，说明训练和推理流程已经打通。

八、Q/K/V 分离对比实验分析

8.1 实验目的

为了理解 Q、K、V 分离设计的必要性，本实验在标准 Transformer 注意力结构基础上进行了对比。标准注意力使用独立的 Q、K、V；对比实验中将 K 和 V 合并，即直接令：

$v = k$

这样模型不再使用独立的 Value 矩阵。

8.2 标准 Q/K/V 结构

标准 Transformer 中，Q、K、V 分别由三个不同线性层生成：

```
q = self.w_q(q)
k = self.w_k(k)
v = self.w_v(v)
```

注意力公式为：

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$$

在这个结构中：

矩阵	作用
Q	表示当前 token 想查询什么信息
K	表示每个 token 用什么特征参与匹配
V	表示每个 token 实际提供什么内容

也就是说，Q 和 K 用于计算注意力权重，V 用于提供被聚合的信息内容。

8.3 K/V 合并实验设置

对比实验中，将原代码：

```
q = self.w_q(q)
k = self.w_k(k)
v = self.w_v(v)
```

修改为：

```
q = self.w_q(q)
```

```
k = self.w_k(k)
v = k
```

此时注意力计算变为：

$$\text{Attention}(Q, K) = \text{softmax}(QK^T / \sqrt{d_k})K$$

也就是说，K 同时承担两个任务：

1. 与 Q 计算相关性；
2. 作为被加权求和的内容。

8.4 实验现象

在 K/V 合并后，模型仍然可以完成前向传播和训练，说明从数学形式上看，注意力机制仍然成立。

但是，相比标准 Q/K/V 分离结构，K/V 合并后的模型训练表现不如标准模型稳定，翻译结果也更容易出现语义不准确的问题。

具体表现为：

1. 模型虽然仍能生成德语句子，但输出内容更容易偏向高频句式。
2. 翻译结果中的语义对应关系较弱。
3. 模型更难准确捕捉源语言 token 与目标语言 token 之间的对应关系。
4. 训练过程中模型表达能力受到限制。

8.5 实验分析

K/V 合并后，模型最大的问题在于匹配空间和内容空间被绑定。

在标准注意力中，K 可以专门用于判断“哪些 token 应该被关注”，V 可以专门用于提供“这些 token 应该传递什么信息”。

但是当 K 和 V 合并后，同一个表示既要负责计算注意力权重，又要作为最终聚合的信息内容。这会使模型难以同时优化两个目标。

在机器翻译任务中，Decoder 生成目标词时，需要先判断源语言中哪些词与当前生成位置相关，再从这些词中提取具体语义信息。Q/K/V 分离可以使这两个过程相对独立，从而提高模型表达能力。

因此，Q、K、V 的分离并不是冗余设计，而是注意力机制能够有效工作的关键结构之一。

8.6 小结

通过 K/V 合并实验可以看出，注意力机制虽然可以在没有独立 V 的情况下运行，但模型表达能力会下降。Q、K、V 分离使模型能够区分“注意力匹配”和“内容传递”，从而更好地完成机器翻译中的语义对齐任务。

九、实验结果总结

本实验成功完成了 Transformer 模型在英德翻译任务上的复现。模型能够完成数据加载、分词、训练、保存 checkpoint 和推理生成。

实验生成了模型文件：

transformer_best.pt

并能够使用该模型进行翻译测试。

测试输入：

I love machine learning.

模型输出：

Ich habe sie nicht .

该结果说明模型已经能够生成德语句，在结构对比实验中，K/V 合并模型虽然能够运行，但由于 Key 同时承担匹配和内容表达两个任务，模型表达能力受到限制。位置编码对比实验表明，不使用位置编码会明显削弱模型对词序的建模能力，而简单绝对位置编码虽然能够提供基本位置信息，但相比正弦余弦位置编码泛化能力较弱。

本实验仍存在以下不足：

1. 数据集规模较小，未使用论文原始 WMT14 English-German 数据集。
2. 训练时间有限，模型尚未充分收敛。
3. 推理阶段使用 greedy decoding，没有使用 beam search。
4. 未计算 BLEU 分数，只进行了简单翻译样例分析。
5. 硬件资源有限，batch size 和训练步数低于论文设置。

后续可以从以下方向改进：

1. 使用 WMT14 English-German 数据集。
2. 增加训练 epoch 数。
3. 使用 joint BPE tokenizer。

4. 使用 beam search 和 length penalty。
5. 使用 sacreBLEU 计算 BLEU 分数。
6. 按 token 数进行 batching，提高训练效率。
7. 保存完整 checkpoint，包括 optimizer 和 scheduler 状态。
8. 在更多句子上进行翻译测试和误差分析。

十、实验结论

本文基于 PyTorch 对《Attention Is All You Need》中的 Transformer 模型进行了复现。实验从数据加载、BPE 分词、模型搭建、mask 构造、损失函数、学习率调度、模型训练、checkpoint 保存到推理测试，完成了一个较完整的机器翻译实验流程。

通过实验可以得出以下结论：

第一，Transformer 的 Encoder-Decoder 结构能够有效完成序列到序列建模任务。模型在训练后可以根据英文输入生成德语输出，说明整体流程是可行的。

第二，Q、K、V 分离是注意力机制中的重要设计。Q 和 K 用于计算注意力分布，V 用于提供实际被聚合的信息内容。如果将 K 和 V 合并，模型虽然仍可运行，但表达能力会下降，语义对齐能力变弱。

第三，位置编码是 Transformer 处理自然语言序列不可缺少的组成部分。Self-Attention 本身不包含顺序信息，必须通过位置编码让模型感知词语的先后顺序。去掉位置编码后，模型更容易出现词序混乱和语义错误。

第四，正弦余弦位置编码相比简单绝对位置编码具有无额外参数、可一定程度外推到更长序列等优点，因此适合作为原始 Transformer 的位置表示方式。

总体而言，本实验成功复现了 Transformer 的核心结构和训练流程，并通过对 Q/K/V 结构和位置编码的分析，加深了对注意力机制、语义对齐和序列建模的理解。虽然由于训练时间和硬件资源限制，当前翻译效果与论文结果仍有差距，但实验达到了理解和复现 Transformer 基本原理的目的。

参考文献

[1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. *Attention Is All You Need*. 2017.

[2] PyTorch Documentation.

[3] Hugging Face Datasets Documentation.

[4] Hugging Face Tokenizers Documentation.

